

Vire, a post-mortem

This is the record of making Vire, a new programming language, in one working session, and the field guide that should travel with it. It is written so the text can be selected and copied. The language is published under the MIT license. The aim of this piece is to leave a developer, or a coding agent, with enough context to read a Vire program, write a new one, explain why the runner behaves the way it does, and refuse the jobs Vire is not built to do.

The original request

The request was not for a tutorial in an existing language. It was for a new type of code. The first constraint was novelty: the result should not be a skin on Java, Python, JavaScript, or C. The second constraint was usefulness: after the novelty was established, the language had to carry the everyday jobs people already spread across those languages. The third constraint was independence: a Vire program should not need Python underneath it, and an application written in Vire should not have to drop into HTML, CSS, and JavaScript just to appear on screen. The fourth constraint was restraint: the runner must not be a virus, must not wander the disk, and must not be described as unhackable, because that claim is false for every program that runs on a general-purpose computer.

Those four constraints pulled against each other all day. A language that is truly new is often hard to use. A language that does everything other languages do tends to become a pile of other languages. A language that draws a window still needs some native runner, because a text file cannot execute itself on a bare processor. The work below is the compromise that survived those tensions.

What Vire is

A Vire program is a text file whose name ends in `.vire`. Execution starts at the definition named `main` and stops when that definition finishes, or when the window it opened is closed. There is no loose script body. If `main` is missing, the runner stops before doing work. Comments begin with a tilde and run to the end of the line. Blocks close with the word `end`. Assignment is written `let name be expression`. Printing a line is `say`. A definition is introduced with `define`, takes arguments after `with`, and returns a value with `return`. A call looks like `area with 3, 4`. Lists are written in brackets. Maps are written in braces, with a colon between a key and a value. The words `while`, `for`, `if`, `else`, `try`, and `miss` are the control forms. The words `true`, `false`, and `none` are values.

The runner is `vire.exe` on Windows. You pass it one program path. The folder that contains that file becomes the sandbox root. `read`, `write`, `files`, `use`, and `need` are all judged against that folder. A second folder of `source`, `deps`, holds packages. Each package is a directory whose code file is `lib.vire`. A program names a package with `need` and a quoted word. A program names a sibling file with `use` and a quoted relative path that must end in `.vire`.

This shape is deliberate. Agents often invent a main function, a package manager, and

a filesystem in the same hour, and then they reach outside the project. Vire makes the legal moves look like the syntax, and it makes the illegal moves fail with a short message.

How the session actually went

The first design was not the language described above. It was a model called Silt, in which a program was a bed of grains and execution was a rising flood. A grain had a hardness. When the water reached that hardness, the grain washed away and its action ran. Actions could lay new grains at future flood levels. The idea was genuinely different from statement order, and a small interpreter ran. It was set aside because the person who asked for the language recognized, correctly, that it was not yet a programming language in the sense of Java or the other languages they already use. Novelty that cannot express an ordinary loop in an ordinary way does not meet the request.

The second design was Vire as an imperative language, still hosted by Python. It grew definitions, arithmetic, strings, booleans, while, if, and calls. Then it grew lists, maps, for, try, files, and a set of operations: length, push, slice, keys, join, and the rest. The objection at that point was also correct. If the way you run a program is `python vire.py`, then Python is the language and Vire is a script. The Python file was deleted. The implementation moved into a Windows executable named `vire.exe`. From then on, the command is `vire.exe` and a program file.

Widening the executable is where the language became something a person could keep. Local files gained a hard boundary. Packages arrived as need. A page builder and a local server arrived so a program could write a site and open it at 127.0.0.1 on a high port. A sample notes page proved the server. That success created the next objection: the page was HTML, the look was CSS, and the save button was JavaScript. The application was still borrowing three other languages. The response was a window vocabulary that belongs to Vire: open, line, field, button, value, and show. `work.vire` uses only those words, plus write, to put a note on disk. `vire.exe` draws the window. The browser path still exists, because browsers read HTML, but it is no longer the definition of a Vire application.

The last stretch of the session was documentation. `GUIDE.md` is the manual an agent should read before editing a program. This article is the narrative and the caution list. Both belong in the public repository so the language is not trapped in a chat transcript.

The syntax, in the order a newcomer needs it

A smallest program is three lines. define main, then say and a quoted sentence, then end. Run it with `vire.exe hello.vire` from the folder that contains the executable, or pass a full path to the program. The runner prints the sentence and exits. That is the hello-world test. If it fails, fix the runner before inventing features.

Names are created with let. let n be 1 stores a whole number. let who be "ada" stores text. let ready be true stores a boolean. let empty be none stores the empty value. A

later let of the same name in the same definition replaces the value. Text uses double quotes. The escapes are quote, backslash, newline, and tab. Numbers may include a decimal point. When both sides of a division are whole numbers, the result is whole-number division, not a fraction. That surprise is recorded here so nobody treats slash as a calculator key without testing it.

Arithmetic uses plus, minus, star, slash, and percent. Comparisons use the usual two-character forms, plus the single character forms for less and greater. and, or, and not combine tests. and and or evaluate the right side only when the left side does not already decide the result. Parentheses group expressions. Plus also joins text to text, and it joins two lists. Plus does not silently mix a number and a list.

The sharp edge that cost real debugging time is argument extent. A call consumes the following expression. escape with title plus a closing tag does not mean escape the title and then add the tag. It means escape the title joined with the tag, so the tag is escaped too and the markup breaks. The repair is parentheses around the call. Say hi, then a space, then text with n, is written with the call in parentheses. Any agent that generates Vire should parenthesize every call that sits beside plus.

if tests a value, runs a block, and may continue with else if or else before end. Empty text, none, false, zero, an empty list, and an empty map are all untrue. Other values are true. while repeats until the test is untrue, and it refuses to spin past one hundred thousand passes. for walks a list, the characters of a text, or the keys of a map. skip starts the next pass. stop leaves the loop. Both words are errors outside a loop. A whole program also stops if it performs more than one million steps, so a nested explosion still ends.

define names a body. Parameters follow with, separated by commas. return sends a value back and leaves the definition. If the body ends without return, the call yields zero. Calls may not nest more than sixty-four deep, which blocks a recursive accident from consuming the machine. Each definition has its own names. There is no shared mutable global table. If two files define the same name, loading fails. That is strict on purpose. Silent overwrite is how agents corrupt a program they only half read.

Lists are ordered and mutable. push adds at the end and returns the same list. drop removes the last item and returns it, and it fails on an empty list. put value into name at index writes a slot. at reads a slot. Negative indexes count from the end. An index outside the list is an error, not a wrapped surprise, except for the negative form that is defined as from the end. slice copies a half-open range and tolerates bounds by clipping. span with 1, 3 builds the list of whole numbers from one through three, and it refuses a span that would blow the size cap. sort copies a list of numbers into ascending order. Maps store text keys. A number used as a key is shown as text. has asks whether a key exists. keys lists the keys. put writes a map entry the same way it writes a list slot. kind reports number, text, list, map, bool, or none. text and number convert. number fails with a catchable error when the text is not numeric. lower and upper change letter case. split and join move between text and lists. find returns an index or negative one.

try runs a block. miss runs when that block raises a Vire error. The name after miss receives the message. The handler does not catch a malformed program, because a malformed program never starts. Use try around number, read, fetch, and any index you do not fully trust.

Files, packages, data, and the clock

write with a relative path and a value stores text. It creates missing directories under the sandbox root. read loads that text back. files lists names in a directory, not full machine paths, and not a recursive walk of the disk. Absolute paths are refused. A path that contains dot-dot is refused. The runner will not write an extension used to launch a program: exe, dll, bat, cmd, ps1, com, scr, vbs, or msi. A file larger than one million characters is refused in either direction. These rules are the difference between a sample that saves a note and a sample that replaces the runner or reads another user's folder.

json turns a Vire value into one text document. parse turns that text back into a value. The subset is objects, arrays, strings, numbers, true, false, and null. null becomes none. This is a data notation, not a second programming language, and it is how work that must reload tomorrow should be stored. now is the local time as a year-month-day hour-minute-second string. In an expression, write it in parentheses, because a bare name is otherwise looked up as a stored value. The runner treats the bare name now as the clock when it is used as a value.

fetch performs one HTTP GET. The address must begin with http:// or https://. The wait is at most five seconds. The body is at most one million bytes. Redirects are limited. Failure becomes a Vire error that try can catch. fetch is the seam where Vire touches the outside world, so it is narrow on purpose. There is no general socket API, no upload of the sandbox, and no shell.

need loads deps, then a package word, then lib.vire. The word may contain letters, digits, and underscores, and nothing else. use loads a sibling or nested .vire file by relative path. At most thirty-two files may be pulled in. A cycle is an error. A duplicate definition name is an error. A package does not need its own main. Only the file you pass to vire.exe needs main. The ui package in the repository shows the pattern: small definitions, no main, consumed by a program that does have main.

The window

open with a title creates the window. line adds a label. field adds a text box and remembers it under the name you give. button takes a label and the name of a definition to call. value reads a field. show displays the window and waits until the person closes it. The click handler runs on the same runner, so the definition it names can write files, compute, and even add another line. If that definition fails, the window shows the message instead of killing the process silently.

work.vire is the canonical application. It opens a window titled Vire tasks, shows two lines of status, offers a note field, and saves the field to data/note.txt when Save is pressed. The entire source is Vire. There is no stylesheet and no script file

beside it. If you are an agent asked to build an app, start from that file and change the lines, the field name, and the save definition. Do not invent a parallel HTML app unless the person asked for a browser page.

The local site, and why it is secondary

serve with a port from 1024 through 65535 listens on 127.0.0.1 only. It serves the site directory next to the program. It accepts GET and HEAD. It allows html, css, js, svg, json, and txt. It refuses dot-dot and it refuses to treat a program file as content. Ctrl+C stops it. This exists because a browser is a separate program with its own languages. Vire can prepare files for that browser and can hand them out on the loopback interface. That is useful. It is not the same claim as Vire being the language of the page. When the requirement is that no other language appear in the application source, use the window, not serve.

An agent building a site should still escape text that came from a person. The escape operation replaces the characters that would break markup. Remember the parentheses rule, or the closing tags will be escaped along with the title. That bug happened during the session, was visible in the served page, and was fixed by parenthesizing the call. It is the example to remember.

Use cases that match the runner we actually shipped

One, print a count or a total, the way count.vire walks from one to five. Two, convert a unit inside a definition that takes a number and returns a number. Three, ask a question on the terminal and branch with if. Four, store a person or a record in a map and read one field with at. Five, walk scores, skip blanks, and add the rest. Six, clean a line of words with split, lower, join, and sort. Seven, save a note and read it back, which is the file test. Eight, share a helper through need so two programs do not copy it. Nine, wrap a risky conversion in try and print the problem from miss. Ten, open a small desktop tool with open, field, button, and show. Eleven, store a list of tasks as json and reload it. Twelve, build a loopback page only when a browser is truly required.

If a proposed use case needs a microphone, a game engine, a phone store, a kernel driver, or an account on someone else's computer, it is outside the language as shipped. Say so, and offer the nearest thing inside the list above.

What an agent should do, step by step

Read GUIDE.md once. Then read the program you were asked to change, plus any file named by use or need. Do not scan the whole disk. Create or edit only .vire files and the data files those programs are supposed to own. Keep paths relative. Include define main in the file that will be executed. Do not put main in a library unless that library is itself a program. Parenthesize calls that are combined with plus. After editing, run vire.exe on the program and read the output. A window program stays running until the window closes, so do not start it and then assume the terminal has hung for a mysterious reason. If you change Vire.cs, rebuild vire.exe with the C# compiler command in the guide before you trust a run. The command

references the Windows forms and drawing libraries because the window is part of the runner. Do not add a feature that starts a shell, binds every network interface, or writes an executable into the sandbox. Those were rejected on purpose.

When a run fails, the message starts with vire: and usually includes a line number. Missing main, a bad path, a missing definition, a type mismatch, and a refused write are the common ones. Fix the program. Do not weaken the runner to make a sample pass.

What we refused, and why the refusal is part of the design

We refused to call the runner unhackable. The accurate sentence is that a program is boxed into its folder, capped in time and size, and unable to launch other programs. We refused to keep the Python interpreter beside the executable, because two runners means nobody knows which language is real. We refused to let write replace vire.exe or drop a batch file. We refused to let serve listen on the open network. We refused to treat HTML as the only application story after the requirement was restated. We also refused the fantasy that one session can absorb every feature of every other language. Device drivers, application stores, and a public package index are products, not syntax. The repository can grow toward more of the everyday jobs. It should grow by adding Vire definitions and careful runner features, not by embedding another language and calling the combination Vire.

Known limits a careful reader should memorize

Whole-number division. Call expressions that swallow the following plus. No global variables. No user-defined types beyond maps and lists. No concurrent definitions. The window blocks until it is closed, so a program does not continue after show until the person is done. serve blocks until Ctrl+C. fetch does not post a body. JSON strings in the writer escape quotes and backslashes, and the reader accepts a practical subset, not every corner of the JSON specification. The step counter counts work, so a server that is actively handling a flood of requests can eventually trip a limit that an idle design might not. The sandbox judges paths lexically and by full path prefix. It is a seatbelt, not a formal proof. None of these are secrets. They are the contract.

How to judge whether a change belongs

A change belongs if a Vire program can do a new everyday job without a second programming language in the program text, and if the change still respects the folder boundary and the refusal to launch other programs. A change does not belong if it exists only to make Vire look like it contains C, Python, and JavaScript at once. The language is already one language. Adding a foreign source language in the middle of a program would undo the point of the session.

The repository

The public repository contains Vire.cs, the guide, the samples, the ui package, the MIT license, and this article as a PDF. Samples include the counter, the greeting and function demo, the guard that tries to escape the folder, the limit demo, and the

window application. Build instructions live in the guide. The license allows use, modification, and redistribution. It does not include a warranty. If you ship Vire to other people, keep the license notice with the source.

A closing account

At the start of the session there was no Vire. There was a request for a language that did not exist, then a correction that it had to be a real programming language, then a correction that it had to be broad, then a correction that it must not be a virus, then a correction that it must not be hosted by Python, then a correction that an application must not lean on HTML, CSS, and JavaScript to count. Each correction narrowed the honest claim and improved the artifact. The artifact that remains is a runner, a syntax, a sandbox, a package convention, a window, a local file and data story, an optional loopback site, and a manual.

That is enough to play with and enough to grow. It is not enough to retire every other language, and this post-mortem will not pretend otherwise. Growth looks like new .vire files, new packages under deps, and runner changes that keep the same boundaries. The test of a growth step is simple. A newcomer should still be able to read define main, run vire.exe, and predict where a file will be written. If a change breaks that prediction, it is not a feature. It is a different language, and this one already cost a full session to make coherent.

A worked transcript for the first hour with the repository

Clone the repository, or copy the folder, onto a Windows machine. Open a terminal in that folder. Run `vire.exe count.vire`. You should see the lines 1, 2, 3, 4, and 5. Run `vire.exe greet.vire`. You should see a greeting, the number 42, and the line it ran. Those two runs prove definitions, arithmetic, loops, calls, and branching before you touch the window. Run `vire.exe guard.vire`. You should see two refusals, both saying that files stay inside the program folder. That is the security test, and it should keep failing in exactly that way. If a future patch makes `guard.vire` print a secret from another directory, revert the patch.

Then run `vire.exe work.vire`. A window opens. The title is Vire tasks. Two lines of text are already there. The box is empty. Type a sentence and press Save. A new line, Saved, appears in the window. Close the window. Open `data/note.txt`. The sentence is there. If the file is missing, the sandbox root is not the folder you think it is, or write was pointed at a forbidden extension. Do not fix that by disabling the check. Fix the path.

If you are extending the language rather than a program, change `Vire.cs` and rebuild with the compiler command in `GUIDE.md` before you rerun the four programs above. A rebuild that is skipped is the usual reason a new word is reported as an unknown definition. The executable and the source had diverged once during this session for that exact reason, when a rebuild could not overwrite a running `vire.exe`. Stop the old process, then compile.

What the samples are for

count.vire is the control-flow baseline. greet.vire is the definition baseline, including a returned value used in a comparison. everything.vire, if you still have it from the working tree, is the crowded feature pass: lists, maps, skip, kind, join, read, write, and miss. limits.vire shows a refused program-file write and a loop that is cut off. baduse.vire shows that use will not open a text file as code. app.vire and math.vire show use across files. web.vire and serve.vire show the browser path and should be treated as optional. work.vire is the application you should copy. deps/ui/lib.vire is the package you should copy when a helper is shared. If a sample and this article disagree, trust the sample that you just ran, then fix the article. A manual that drifts from the runner is worse than a short manual.

Advice for the post that points at the repository

Say that Vire is a new language with its own syntax and a Windows runner. Say that programs are .vire files. Say that the interesting application is a window written only in Vire, and that files stay in the program folder. Say that the license is MIT. Do not say that it replaces every other language. Do not say that it cannot be abused if a person rewrites the runner. Do not paste a drive letter from a private machine into the post. Link the repository, and let GUIDE.md do the teaching. The image that accompanies the post should show a single window and a page of Vire source, not a collage of other languages' logos. The language spent the whole session climbing out of other languages. The picture should not put it back.

A note on length and honesty

This account is long because the decisions were easy to flatten into a slogan and the slogan would be false. The short version, if you need it in conversation, is this: Vire is a small language you run with vire.exe, it can compute, store files, load local packages, and open a window, and it refuses to leave its folder or launch other programs. The long version is everything above, including the flood-language detour, the Python runner we deleted, the HTML detour we demoted, and the parentheses bug that escaped a closing tag. Keep the long version with the code. Future changes will be easier to judge against a record that admits what failed.

End of record.